

## Reasons you should use Kotlin

One of the first questions that developers ask themselves while selecting a programming language is, “Why use this language?” Here are some important reasons for why you should use Kotlin.

- *It is multi-platform:* Kotlin can be used to build applications that target the following environments:
  - Native code
  - In-browser execution (through JavaScript trans-compilation)
  - Android environment
  - Java Virtual Machine
- *It is concise:* One of the main advantages of using Kotlin is its ability to reduce the amount of boilerplate code considerably. So developers can focus on the task at hand, rather than worrying about the skeleton in which the code needs to be placed.
- *It is safe:* Kotlin has lots of inbuilt mechanisms to prevent a large class of errors. One that will be liked by many developers is ‘null pointer exceptions’.
- *It is interoperable:* It is a major advantage to be able to use libraries across JVM, Android and JavaScript environments.
- *It is tool-friendly:* For many developers, the integrated development environment (IDE) is a factor that affects their productivity. Kotlin can be used with many popular IDEs (IntelliJ IDEA, Android Studio, Eclipse, etc). You can interact with it through the command line as well.

## Why Kotlin has been given this name

Kotlin is the name of an island near St Petersburg in Russia. The Kotlin team chose this name since the name ‘Java’ also has a link to an island (apart from the connection with coffee).

With the release of Android Studio 3.0, Kotlin is completely supported as a programming language for the Android OS. It can be used as an alternative to the standard Java compiler.

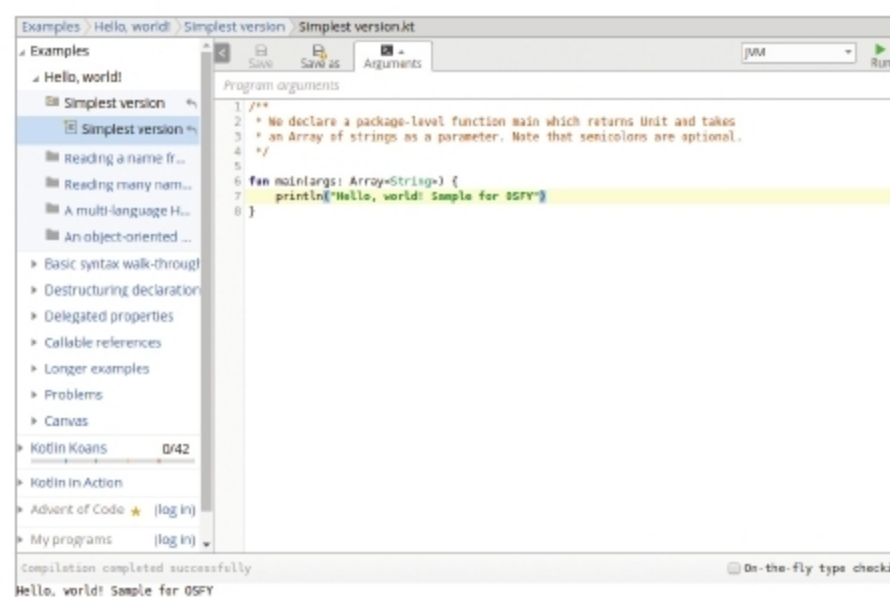


Figure 2: Kotlin – try it out online

## Getting started

The easiest way to get started with Kotlin is to use <https://try.kotlinlang.org/>. The environment provided here is intuitive. There are lots of sample programs provided in this environment. Let's first write the ‘Hello World’ program, as follows:

```
fun main(args: Array<String>) {
    println("Hello, world! Sample for OSFY")
}
```

After writing this code, select one of the following as the target:

- JVM
- JavaScript
- JUnit

Then click on the *Run* icon. The output appears in the *output* window.

The input can be read from command line arguments. A sample code snippet is as follows:

```
fun main(args: Array<String>) {
    if (args.size == 0) {
        println("Please provide a name as a command-line argument")
        return
    }
    println("Hello, ${args[0]}!")
}
```

It can be observed from the above code snippet that the semi-colons are optional in Kotlin. The newline is automatically inferred by Kotlin.

Multiple arguments can be read from the command line using ‘for’, as shown in the following code snippet:

```
fun main(args: Array<String>) {
    for (name in args)
        println("Hello, $name!")
}
```

If you want the object-oriented version of *Hello World*, here is the code snippet:

```
class Greeter(val name: String) {
    fun greet() {
        println("Hello, ${name}");
    }
}

fun main(args: Array<String>) {
    Greeter(args[0]).greet()
}
```

It may be observed that the ‘new’ keyword is not used for creating an object.